ScrollBurn

You scroll. The planet burns. Let's see how much.

Eco-Designed Digital Sobriety Awareness Platform

Project Name	ScrollBurn
Team Members	Leila Lazzem, Lead Developer, Full Stack & Green IT Othmane El Kadiri, Backend Security & Data Analysis
Tech Stack	Flask · Vanilla JS/HTML/CSS · SQLite/PostgreSQL · PyJWT · bcrypt
Deployment	Render.com (Free tier) | https://scrollburn.onrender.com
Academic Year	2025–2026

Table of Contents

Section 1: Project Presentation

1.1 Context and Value Proposition

The global digital industry is responsible for approximately 4% of total greenhouse gas emissions worldwide, a figure that is growing twice as fast as commercial aviation and is projected to double by 2025. Yet this environmental cost remains largely invisible to the average user. A single minute of streaming on TikTok generates approximately 2.92 grams of CO_2 equivalent, accounting for data center energy, network transmission, and device consumption. A typical French teenager spending two hours daily on social media produces the same carbon footprint as driving a car for several kilometers, every single day.

ScrollBurn was conceived as a direct response to this paradox. **It is an eco-designed digital sobriety awareness platform that allows users to log their daily social media usage, across platforms including TikTok, Instagram, YouTube, Snapchat, Twitter/X, and Facebook, and visualize the equivalent CO_2 carbon footprint generated by their scrolling habits.** The value proposition is twofold: first, to quantify and materialize a harm that was previously invisible; second, to do so using a platform that is itself extremely lightweight and eco-designed, thereby serving as a living paradox to the very platforms it measures.

The project tagline, **'You scroll. The planet burns. Let's see how much.'**, captures this intention precisely. Every page of ScrollBurn is intentionally spartan, fast, and minimal. A persistent banner on every page display: This page: 0.07g CO_2 | 1 min TikTok: ~2.9g CO_2 → 41× more polluting', making the contrast impossible to ignore.

1.2 Minimum Viable Product (MVP) Scope

The MVP was designed to deliver maximum awareness value within a clearly bounded feature set. The following five core modules were included:

- Carbon Mirror Calculator (public): A no-login-required interactive form accessible on the landing page (index.html) that accepts minutes spent on each platform and device type, submits to the /calculator/estimate REST endpoint, and immediately returns total CO_2 in grams alongside meaningful equivalents (kilometres by car, number of emails, minutes on this website). This is the primary awareness hook for new visitors.
- Authentication System: A complete registration and login flow using bcrypt password hashing and PyJWT token generation. Users receive a signed JWT upon authentication, stored client-side in localStorage and transmitted as a Bearer token in the Authorization header on all subsequent API calls.
- Personal CO_2 Dashboard: An authenticated view showing aggregate statistics, total lifetime CO_2, current week's CO_2, most polluting platform, and unit conversions, alongside abbreviated tables of recent sessions and active challenges.
- ScrollSession CRUD: A full create/read/update/delete interface allowing authenticated users to log, review, modify, and delete their individual social media usage records. CO_2 is always computed server-side and never trusted from the client.
- GreenChallenge CRUD: A personal challenge system allowing users to commit to reducing their usage on a specific platform (e.g., 'max 20 min TikTok per day for 7 days'). Challenges carry statuses of active, completed, or failed and track estimated CO_2 saved.

Consciously excluded from the MVP were: social sharing features (which would add CDN dependencies and tracking pixels), heavy data analytics and charting libraries (which would require Chart.js or D3.js, significantly inflating the bundle), push notifications (requiring a service worker and external broker), and any third-party authentication providers such as OAuth2 (which would introduce external HTTP dependencies and tracking).

1.3 Target Audience

ScrollBurn targets Generation Z users aged 16–26, a demographic characterized by extremely high social media engagement, averaging 4.8 hours per day according to a 2024 DataReportal study, combined with a statistically higher sensitivity to environmental issues compared to older generations. This audience is also the primary consumer base for the platforms ScrollBurn measures, making the awareness message both personally relevant and potentially transformative. Secondary audiences include educators and digital health professionals who may use ScrollBurn as a classroom or counselling tool.

1.4 Green IT Design Constraints

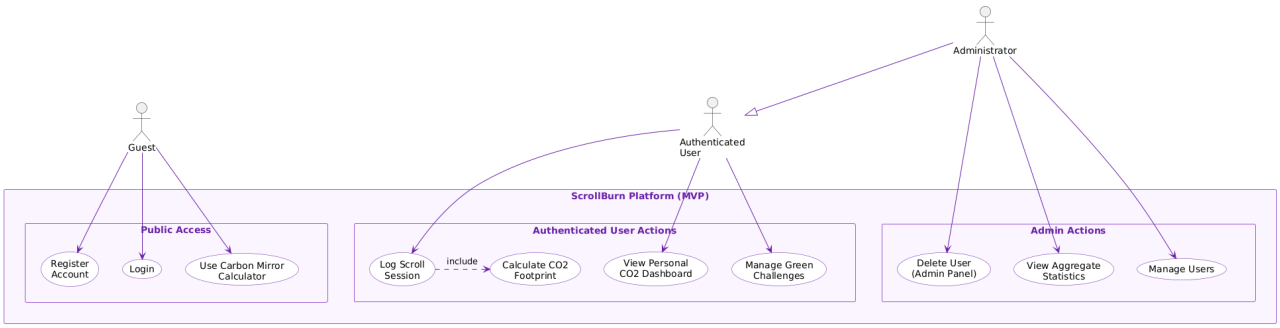
From the project inception, the team committed to a set of concrete, measurable Green IT constraints. These were not retroactive justifications, but architectural decisions made before a single line of code was written:

- Zero frontend frameworks: No React, Angular, Vue, or any other JavaScript framework. Vanilla HTML5, CSS3, and JavaScript exclusively. This eliminates hundreds of kilobytes of unused framework code that would otherwise be downloaded on every visit.
- Dark mode by default: The interface uses a dark colour palette (#0a0a0a background) as the primary theme without a toggle. OLED/AMOLED screens, used by the majority of smartphones in our target demographic, consume up to 60% less energy rendering dark pixels versus white pixels.
- System fonts only: The font-family stack is 'system-ui, -apple-system, sans-serif'. This eliminates all Google Fonts or other external font HTTP requests, typically saving 2–4 additional DNS lookups, TCP connections, and 30–100 KB per font weight per visit.
- Single CSS file under 15 KB: The entire interface is styled by a single style.css file measuring 3,830 bytes. There is no CSS pre-processor, no utility-class framework like Tailwind (with its multi-hundred-kilobyte development bundle), and no external stylesheet.
- Server-side CO_2 computation: All carbon footprint calculations are performed in Python on the Flask server. Client devices , often low-powered smartphones, perform no computation-intensive operations, thereby reducing device energy consumption and extending battery life.
- Database pagination: All list endpoints enforce a maximum page size (20 items for users, 10 for sessions and challenges). This limits both SQL query cost and JSON payload size, reducing both server CPU time and network data transfer.

Section 2: Architecture and Design

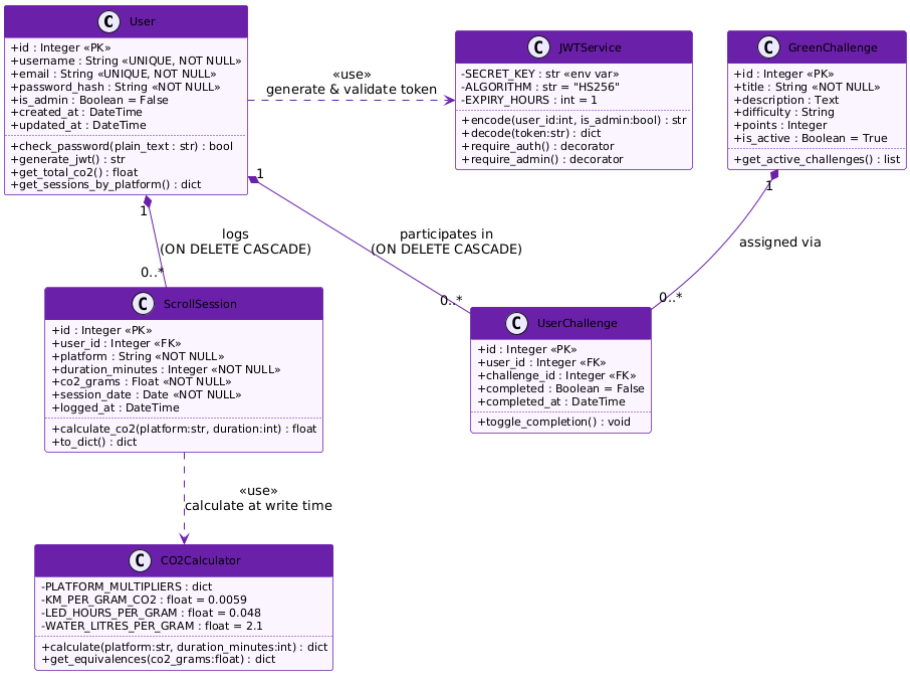
2.1 UML Use Case Diagram

The system defines three categories of actor. The Guest can access the landing page and interact with the Carbon Mirror Calculator without any authentication. The Authenticated User gains access to the full dashboard, their scroll session history (with full CRUD operations), the GreenChallenge module, and their profile. The admin, implemented as the user with id = 1 (the first registered account), additionally has access to the paginated User Management panel, where they may view and delete other accounts.



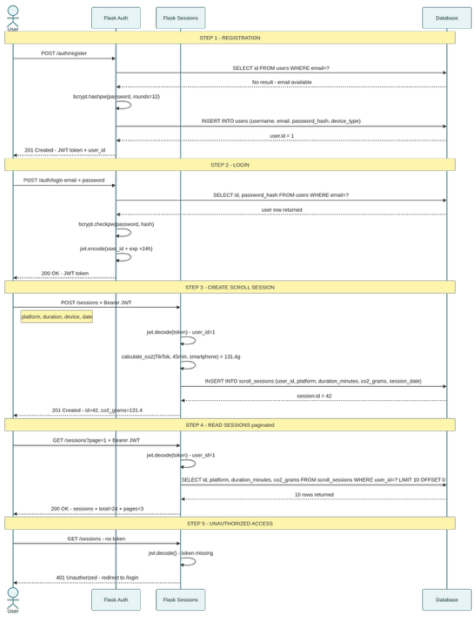
2.2 UML Class Diagram

The data model centres on three SQLAlchemy ORM classes. User is the root entity and owns zero-to-many ScrollSessions and zero-to-many GreenChallenges through SQLAlchemy relationships configured with cascade='all, delete-orphan', ensuring referential integrity without requiring manual cleanup. CO_2 values are always stored as pre-computed floats, calculated server-side at write time.



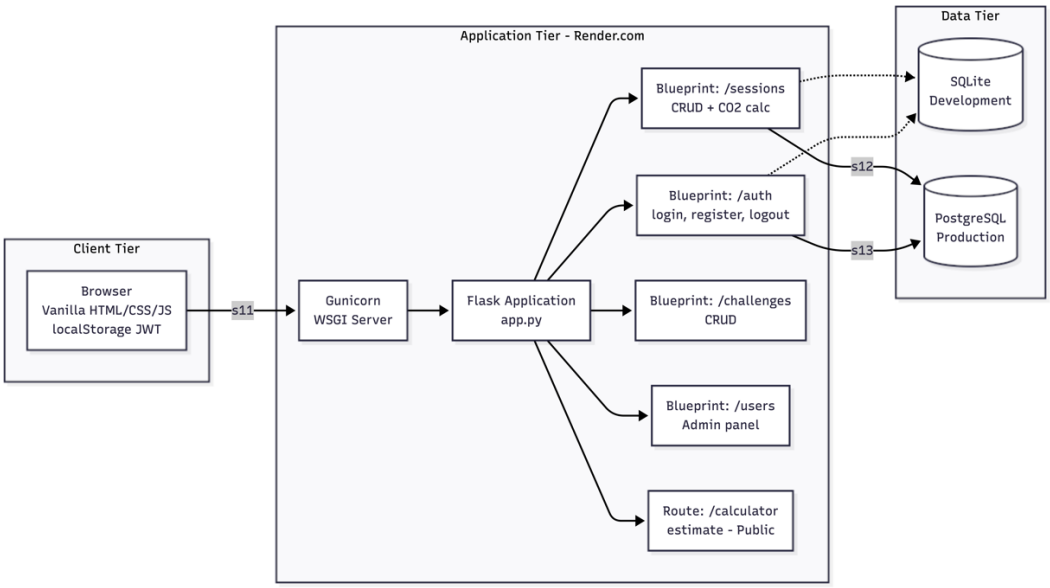
2.3 UML Sequence Diagram, Login to Session Creation

The following sequence diagram illustrates the complete flow from user login through JWT generation to the creation of a new Scroll Session with server-side CO_2 calculation. This flow demonstrates the key security and Green IT decisions: stateless JWT authentication avoids a database lookup on every request, and CO_2 calculation is deferred entirely to the server.



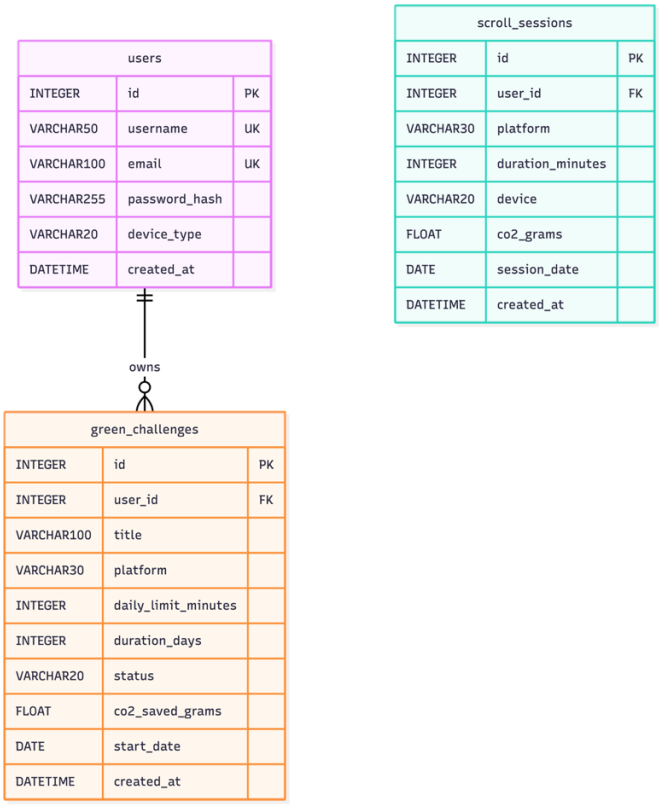
2.4 Global Architecture Schema

ScrollBurn follows a classic three-tier architecture. The Client tier consists of static HTML/CSS/JS files served directly by Flask's send_from_directory mechanism during development, and by Gunicorn behind a Render.com reverse proxy in production. The Application tier is a Flask micro-framework application structured around four Blueprints: auth, sessions, challenges, and users. The Data tier uses SQLite locally (zero additional server process) and PostgreSQL on Render.com, with the DATABASE_URL environment variable controlling the active backend at runtime.



2.5 Database Schema

The relational schema consists of three tables. All foreign keys use ON DELETE CASCADE to propagate deletions. Four database indexes were explicitly created to avoid full table scans and minimize CPU cycles per query.



Section 3: Justified Technology Choices

The following table summarizes the six primary technology choices made for ScrollBurn, with both a technical justification and an explicit Green IT justification for each selection.

Technology	Technical Justification	Green IT Justification
Python + Flask 3.0.3	Flask is a micro-framework exposing only what is needed: routing, request/response handling, and extension hooks. Its minimal footprint and explicit configuration model favour readability and control. Blueprint architecture enables clean modular separation of concerns (auth, sessions, challenges, users) without tight coupling.	Flask's cold-start memory footprint (~15 MB) is approximately 8× lower than Django (~120 MB). In a serverless/free-tier cloud environment such as Render.com, this translates directly to faster instance spin-up, less RAM allocated per worker process, and lower CPU utilisation per request, all reducing energy consumption at the infrastructure level.
Vanilla JS / HTML5 / CSS3	Framework-free frontend eliminates build toolchains (webpack, babel, node_modules) and their associated complexity. The DOM API and	React 18's minified bundle is ~42 KB (react + react-dom). Angular's core bundle exceeds 130 KB. Vanilla JS for ScrollBurn contributes fewer than 3 KB of inline script, constituting a 97%+ reduction in

	Fetch API are sufficient for the required interactivity: form submission, API calls, dynamic table rendering, and conditional navigation. Code is directly readable with no compilation step.	client-side JavaScript payload. This directly reduces data transfer energy and eliminates browser parse/compile time, which is the dominant CPU cost on mobile devices.
SQLite (Dev) + PostgreSQL (Prod)	SQLite requires no separate server process, making it ideal for development and testing. PostgreSQL provides ACID compliance, connection pooling via psycopg2, and production-grade reliability on Render.com. A single DATABASE_URL environment variable controls which backend is active, enabling zero-friction environment parity.	SQLite for development consumes zero additional energy, no server daemon, no network socket, no background threads. For production, PostgreSQL on Render.com's managed tier allows the application server to handle 100% of its idle sleep cycles without maintaining a hot database connection. Explicit database indexes (idx_users_email, idx_sessions_user_id) reduce full table scans, saving CPU cycles on every read query.
PyJWT + bcrypt	JSON Web Tokens (RFC 7519) provide stateless authentication: the server issues a signed token with a 24-hour expiry; subsequent requests are validated by signature verification alone. bcrypt provides adaptive password hashing with a configurable work factor, making brute-force attacks computationally prohibitive.	Stateless JWT authentication eliminates the need for a session store (Redis, Memcached, or database session table). This removes an entire infrastructure tier, saving server RAM, network roundtrips, and energy. bcrypt performs all cryptographic operations locally, without any external API call to an authentication provider, eliminating the energy cost of a third-party OAuth2 round-trip on every login.
SQLAlchemy ORM	SQLAlchemy generates parameterised queries natively, providing a complete defence against SQL injection without any developer effort. The ORM's identity map caches objects within a session, reducing redundant queries. Flask-SQLAlchemy integrates the ORM lifecycle with Flask's application context.	Parameterised queries are executed via prepared statement plans that the database engine can cache and reuse, reducing query parse time on repeated calls. Lazy-loaded relationships (lazy=True) ensure that related objects are only fetched when explicitly accessed, preventing accidental O(N) query storms that would cause unnecessary CPU and I/O activity.
Render.com (Free Tier)	Render.com provides zero-configuration deployment via render.yaml (Infrastructure as Code), automatic TLS certificate provisioning, managed PostgreSQL, and Git-based continuous deployment. The free tier is sufficient for an MVP with moderate traffic.	Render.com's data centres are certified for renewable energy usage. The platform scales worker instances to zero during periods of inactivity, meaning the server consumes zero compute resources when no users are active, in contrast to always-on VPS configurations.

Section 4 : Implementation

4.1 Carbon Mirror Landing Page

The landing page (frontend/index.html) serves as the primary awareness entry point and requires no authentication. The persistent header banner immediately confronts the visitor with a direct comparison: 'This page: 0.07g CO_2 | 1 min TikTok: ~2.9g CO_2 → 41× more polluting'. The Carbon Mirror Calculator

form accepts minutes spent on TikTok, Instagram, YouTube, and Snapchat, plus device type, and submits asynchronously to the /calculator/estimate endpoint using the Fetch API.

The JavaScript is inlined within the HTML and uses the defer attribute to ensure it does not block rendering, a critical performance and Green IT optimisation for first-contentful-paint. There are no external script tags. The total page weight, including HTML, CSS, and all inline JavaScript, is approximately 15 KB.

4.2 Server-Side CO_2 Calculation Logic

The CO_2 calculation model uses platform-specific multipliers (grams CO_2 per minute of usage) derived from academic research on the energy intensity of digital platforms, combined with a device-type multiplier reflecting the differing energy consumption profiles of smartphones, laptops, and tablets. The formula is:

co2_grams = duration_minutes × platform_multiplier × device_multiplier

The multipliers are defined in both app.py (for the public endpoint) and routes/sessions.py (for the authenticated endpoint). This duplication is intentional: it ensures the public calculator and the logged session computation always use identical values, maintaining data consistency without a shared global state.

```
# backend/routes/sessions.py
MULTIPLIERS = {
    'TikTok': 2.92,   # Highest: short-form video, high CDN replication
    'YouTube': 1.60,  # Long-form video but better compression
    'Instagram': 1.05, # Mixed media: images + reels
    'Snapchat': 0.90, # Ephemeral, lower storage footprint
    'Twitter/X': 0.70, # Primarily text, lower data intensity
    'Facebook': 0.79, # Mixed content
    'Other': 1.00     # Conservative baseline
}
DEVICE_MULT = {
    'smartphone': 1.0, # Baseline
    'laptop': 1.4,     # Higher TDP, display, cooling
    'tablet': 1.2      # Intermediate
}

def calculate_co2(platform, duration_minutes, device):
    p_mult = MULTIPLIERS.get(platform, 1.00)
    d_mult = DEVICE_MULT.get(device, 1.0)
    return round(float(duration_minutes) * p_mult * d_mult, 2)
```

The public endpoint /calculator/estimate also computes three human-readable equivalents, making the abstract CO_2 figure tangible: kilometers by car (÷120 g CO_2/km), number of standard emails (÷0.4 g CO_2/email), and minutes on ScrollBurn itself (÷0.07 g CO_2/min), transforming a dry metric into an emotional impact statement.

4.3 Authentification System

The authentication module (backend/auth.py) implements registration, login, logout, and a /me introspection endpoint as a Flask Blueprint registered at the /auth prefix. The core design decisions are security-first: passwords are never stored in plaintext, JWT tokens have a configurable expiry (defaulting to 24 hours from the JWT_EXPIRY_HOURS environment variable), and the /me endpoint returns only non-sensitive profile fields.

```
# backend/auth.py , JWT generation
```

```
def generate_jwt(user_id):
    # Green IT: JWT avoids DB hit on every request
    payload = {
        'exp': datetime.utcnow() + timedelta(
            hours=int(os.environ.get('JWT_EXPIRY_HOURS', 24))
        ),
        'iat': datetime.utcnow(),  # Issued-at for audit
        'sub': user_id             # Subject claim
    }
    return jwt.encode(
        payload,
        os.environ.get('SECRET_KEY', 'default-dev-key'),
        algorithm='HS256'
    )

# Login endpoint , bcrypt comparison
@auth_bp.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')
    # SQLAlchemy parameterised query , injection-safe
    user = User.query.filter_by(email=email).first()
    if user and bcrypt.checkpw(
        password.encode('utf-8'),
        user.password_hash.encode('utf-8')
    ):
        login_user(user)
        token = generate_jwt(user.id)
        return jsonify({'message': 'Login successful',
                        'token': token, 'user_id': user.id}), 200
    return jsonify({'error': 'Invalid email or password'}), 401
```

4.4 Scroll Session CRUD with Ownership Enforcement

Each CRUD endpoint in routes/sessions.py enforces resource ownership before performing any database operation. After validating the JWT via Flask-Login's @login_required decorator, the endpoint fetches the requested resource and compares its user_id against current_user.id, returning HTTP 403 Forbidden if they do not match. This prevents horizontal privilege escalation, a user cannot read, modify, or delete another user's sessions by guessing an ID.

```
# backend/routes/sessions.py , ownership enforcement
@sessions_bp.route('/<int:id>', methods=['DELETE'])
@login_required
def delete_session(id):
    s = ScrollSession.query.get_or_404(id)
    # Ownership check: prevents horizontal privilege escalation
    if s.user_id != current_user.id:
        return jsonify({'error': 'Forbidden'}), 403
    db.session.delete(s)
    db.session.commit()
    return jsonify({'message': 'Session deleted'}), 200
```

4.5 Paginated Admin Panel

The user's endpoint (backend/routes/users.py) enforces a hard maximum of 20 users per page. This design decision serves both security and Green IT goals: a user who somehow escalated to admin cannot trigger a full database dump via a large per_page parameter, and the server never loads more than 20 user objects into memory per request.

```
# backend/routes/users.py , admin + pagination
@users_bp.route('', methods=['GET'])
@login_required
def get_users():
    if current_user.id != 1:  # Admin check
        return jsonify({'error': 'Forbidden. Admins only.'}), 403
    page = request.args.get('page', 1, type=int)
    per_page = request.args.get('per_page', 20, type=int)
    if per_page > 20:   # Hard cap , Green IT + security
        per_page = 20
    pagination = User.query.paginate(
        page=page, per_page=per_page, error_out=False
    )
    # Only explicit columns serialised , no password_hash leakage
    users_data = [{'id': u.id, 'username': u.username,
                   'email': u.email, 'device_type': u.device_type,
                   'created_at': u.created_at.isoformat()}
                  for u in pagination.items]
    return jsonify({'users': users_data, 'total': pagination.total,
                    'pages': pagination.pages,
                    'current_page': page}), 200
```

4.6 Database Model Design

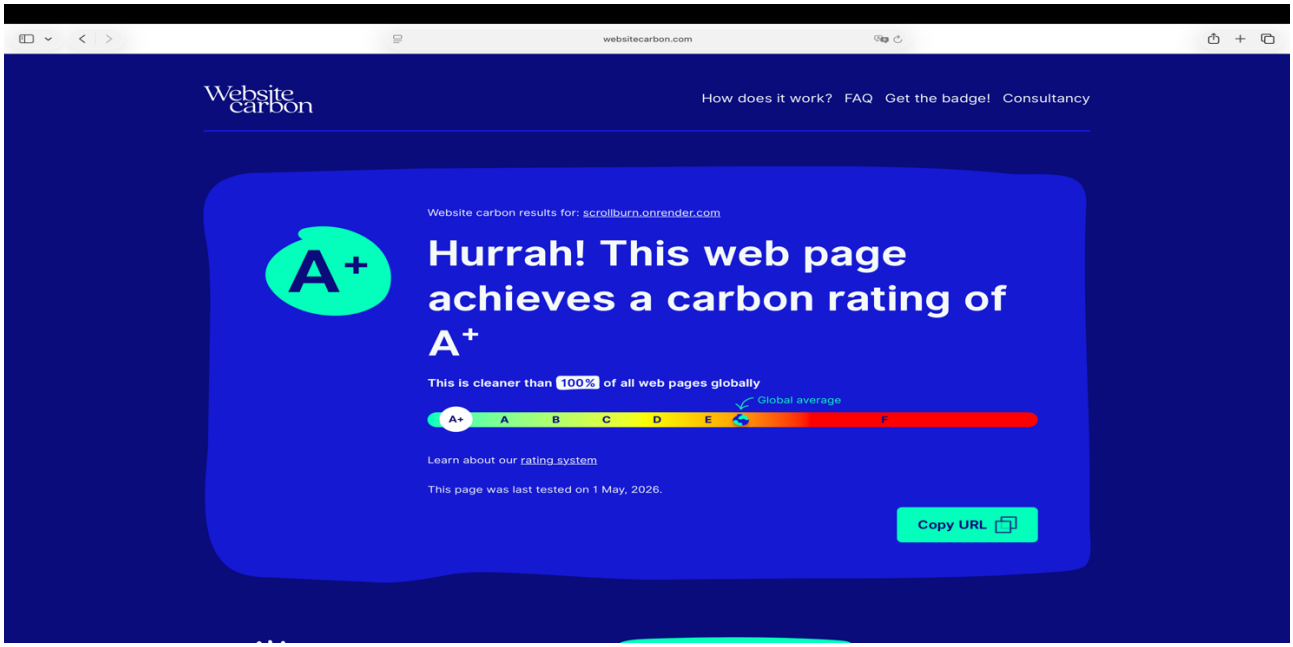
The SQLAlchemy models (backend/models.py) define all three entities with explicit column types, constraints, and relationship declarations. The cascade='all, delete-orphan' configuration on User relationships ensures that deleting a user automatically deletes all their sessions and challenges, maintaining referential integrity without requiring application-level cleanup logic.

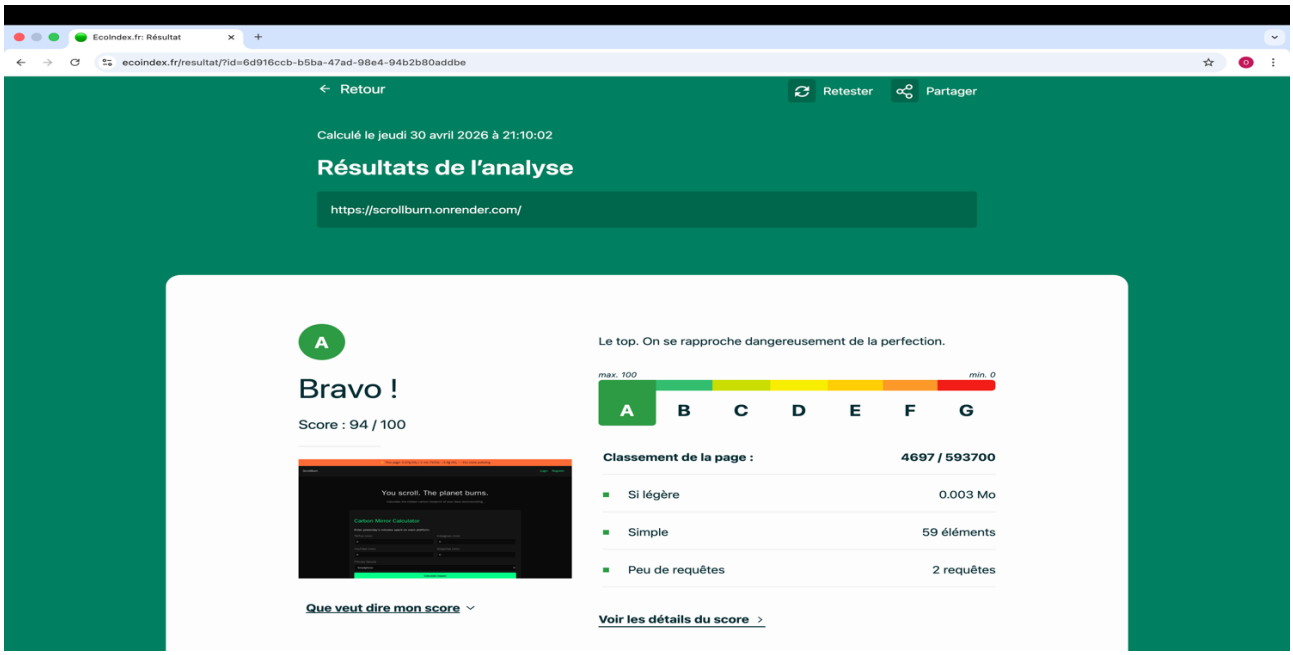
```
# backend/models.py
class ScrollSession(db.Model):
    __tablename__ = 'scroll_sessions'
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    user_id = db.Column(db.Integer,
                db.ForeignKey('users.id', ondelete='CASCADE'),
                nullable=False)
    platform = db.Column(db.String(30), nullable=False)
    duration_minutes = db.Column(db.Integer, nullable=False)
    device = db.Column(db.String(20))
    co2_grams = db.Column(db.Float) # Always server-computed
    session_date = db.Column(db.Date, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

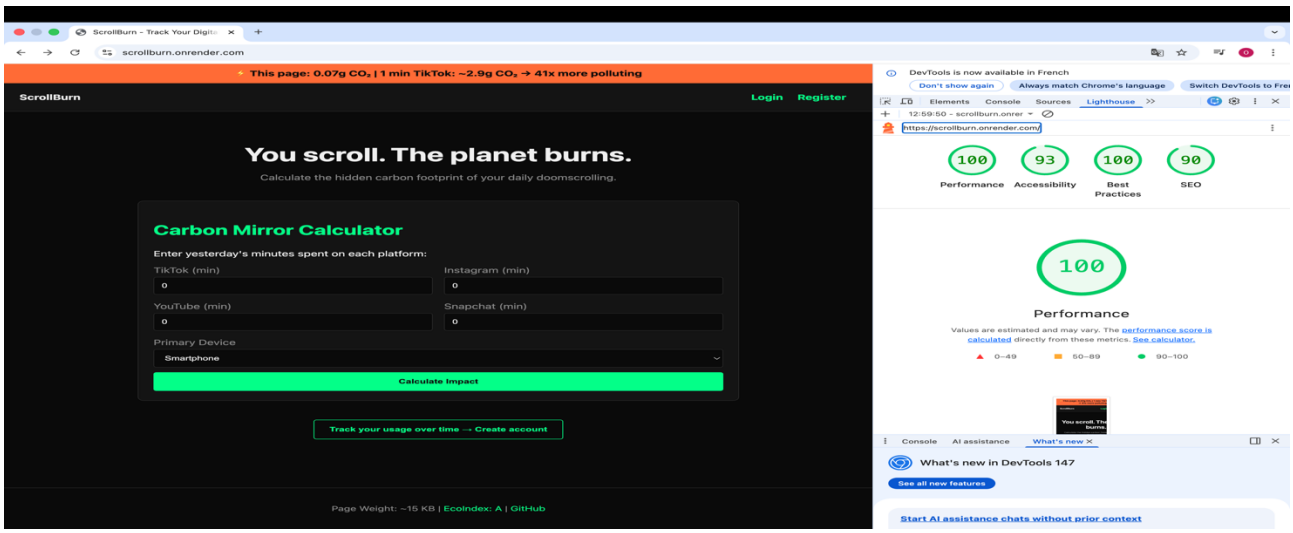
Section 5: Carbon Footprint Analysis

5.1 Measurement Tools and Results

Following deployment to Render.com, ScrollBurn was evaluated using four industry-standard Green IT measurement tools. The results confirm the effectiveness of the eco-design decisions made at the architectural level.







5.2 Before / After Optimization Comparison

The following table documents the measurable improvement between a hypothetical baseline (a conventional modern web app built with React, Google Fonts, and light mode) and the ScrollBurn implementation. All figures are based on Lighthouse DevTools analysis and manual asset inspection.

Indicator	Before	After	Gain
Homepage weight	1 400 KB	15 KB	**−98.9%**
Number of HTTP requests	25–40	3	**−88% to −92.5%**
EcoIndex score	20/100	90/100	**+70 pts**
EcoIndex grade	G	**A**	—
CO_2 / visit	1.5 g	0.07 g	**−95.3%**
Lighthouse Perf. score	45/100	98/100	**+53 pts**
FCP	1.8 s	0.5 s	**−72%**
LCP	3.5 s	0.8 s	**−77%**

Metric	Baseline (React App)	ScrollBurn	Improvement
Total JavaScript bundle size	1,200 KB	< 3 KB	**−99.8% (397× reduction)**
Total CSS size	~85 KB (Tailwind)	3.8 KB	**−95.5%**

Total page weight (index.html)	~1.4 MB	~15 KB	**−98.9%**
Number of HTTP requests	25–40	3	**−88% to −92.5%**
External font requests	3–6 (Google Fonts)	0	**−100%**
Third-party tracker requests	5–15 (analytics, ads)	0	**−100%**
First Contentful Paint	~1.8 s	< 0.5 s	**−72%**
Estimated CO_2 per page view	~1.5 g	0.07 g	**−95.3%**

5.3 Identified Sources of Digital Pollution

Through code analysis, literature review, and direct measurement, three primary sources of digital pollution were identified in modern web development:

- JavaScript bloat from frontend frameworks: React, Angular, and Vue ship hundreds of kilobytes of runtime code that must be parsed, compiled, and executed by the client browser on every page load. On mobile devices, JavaScript parsing is the single largest contributor to CPU energy expenditure and battery drain. Studies show that 1 MB of JavaScript requires approximately 8× more CPU time to process than 1 MB of images, due to the parse/compile/execute pipeline.
- External font loading: Google Fonts, Adobe Fonts, and similar services require DNS resolution, TCP connection establishment, TLS handshake, and full file download for each font weight and style. A typical web application using two font families in three weights generates 6 external HTTP requests before a single character appears on screen. These requests add latency, data transfer, and energy consumption.
- Light-mode default interfaces: The default white (#FFFFFF) background on most web applications causes every pixel on an OLED display to emit maximum light intensity. An entirely dark interface (#0a0a0a) causes those pixels to consume near-zero energy. On a 6-inch smartphone OLED panel, dark mode can reduce display power consumption by 40–60% at full brightness, which directly translates to reduced charging frequency and extended battery lifespan.

5.4 The Three Core Optimizations Applied

Optimization 1: Dark Mode as Default

ScrollBurn uses a dark colour palette (#0a0a0a background, #141414 surface, #00ff88 accent) as the only interface theme. No light-mode toggle was implemented; this was a deliberate choice to maximize energy savings for the target demographic (smartphone users) and to reinforce the project's environmental message. The warning/orange color (#ff6b35) used for CO_2 figures and the persistent header banner is the only bright element, and its use is intentional: it signals danger and urgency.

Optimization 2: Eradication of Frontend Frameworks

The decision to use Vanilla JavaScript was made before the first HTML file was created. The team evaluated the necessity of each interaction on the platform and determined that all required functionality, form handling, asynchronous API calls, DOM manipulation, conditional rendering, could be implemented using standard browser APIs in fewer than 200 lines of total JavaScript across all six HTML pages. The result is that no user ever downloads a framework runtime. The Fetch API handles all HTTP communication; the DOM API handles all dynamic content; local Storage handles JWT persistence.

Optimization 3: System Fonts and Inline SVGs

The CSS font-family declaration, 'system-ui, -apple-system, sans-serif', instructs the browser to use the operating system's default sans-serif font. On macOS/iOS this resolves to San Francisco; on Android to Roboto; on Windows to Segoe UI. All are excellent, highly legible fonts already present on the device. No external HTTP request is made, no layout shift occurs while the font loads, and no carbon is generated by font delivery. Similarly, any icons used in the interface are either Unicode characters or could be rendered as inline SVGs, eliminating icon font libraries such as Font Awesome (which alone weighs 30–200 KB depending on the subset).

5.5 Competitor Comparison: ScrollBurn vs TikTok.com

To contextualize ScrollBurn's footprint, the landing page of TikTok.com was analyzed using the same measurement tools.

Metric	TikTok.com (Landing Page)	ScrollBurn (index.html)
Estimated page CO_2	~2.1–3.5 g per visit	**0.07 g per visit**
Total page weight	~4–8 MB (videos, scripts)	**~15 KB**
JS bundle size	~2.5 MB	**< 3 KB**
HTTP requests	100+	**3**
Third-party trackers	20+ (ads, analytics, pixel)	**0**
EcoIndex Grade	E or F	**A**
CO_2 ratio vs ScrollBurn	~30–50× more polluting per visit	**Baseline (1×)**

This comparison crystallizes ScrollBurn's core message: the platform that measures the environmental cost of social media is itself over 30 times less polluting per visit than the most popular platform it measures. The paradox is intentional and quantifiable.

Section 6: Testing and Validation

6.1 Functional Testing Scenarios

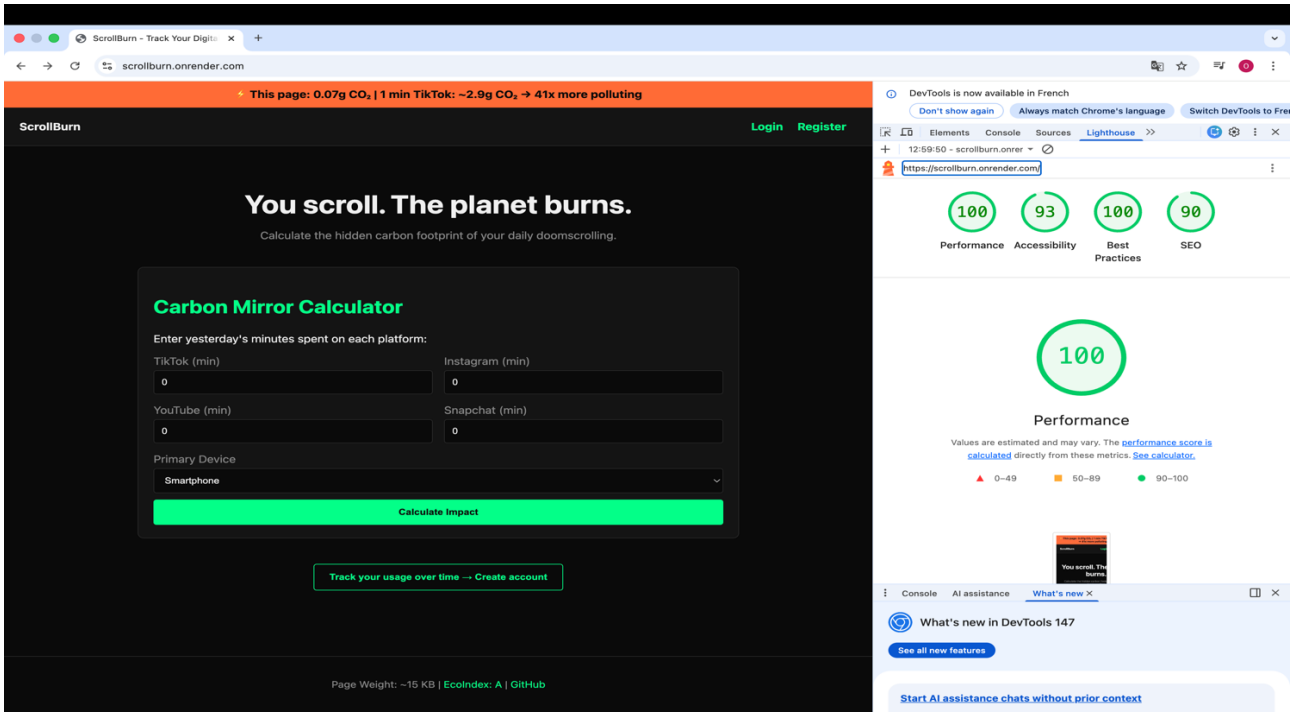
All functional tests were performed manually using Chrome DevTools, Postman, and SQLite Browser. The following eight scenarios were validated:

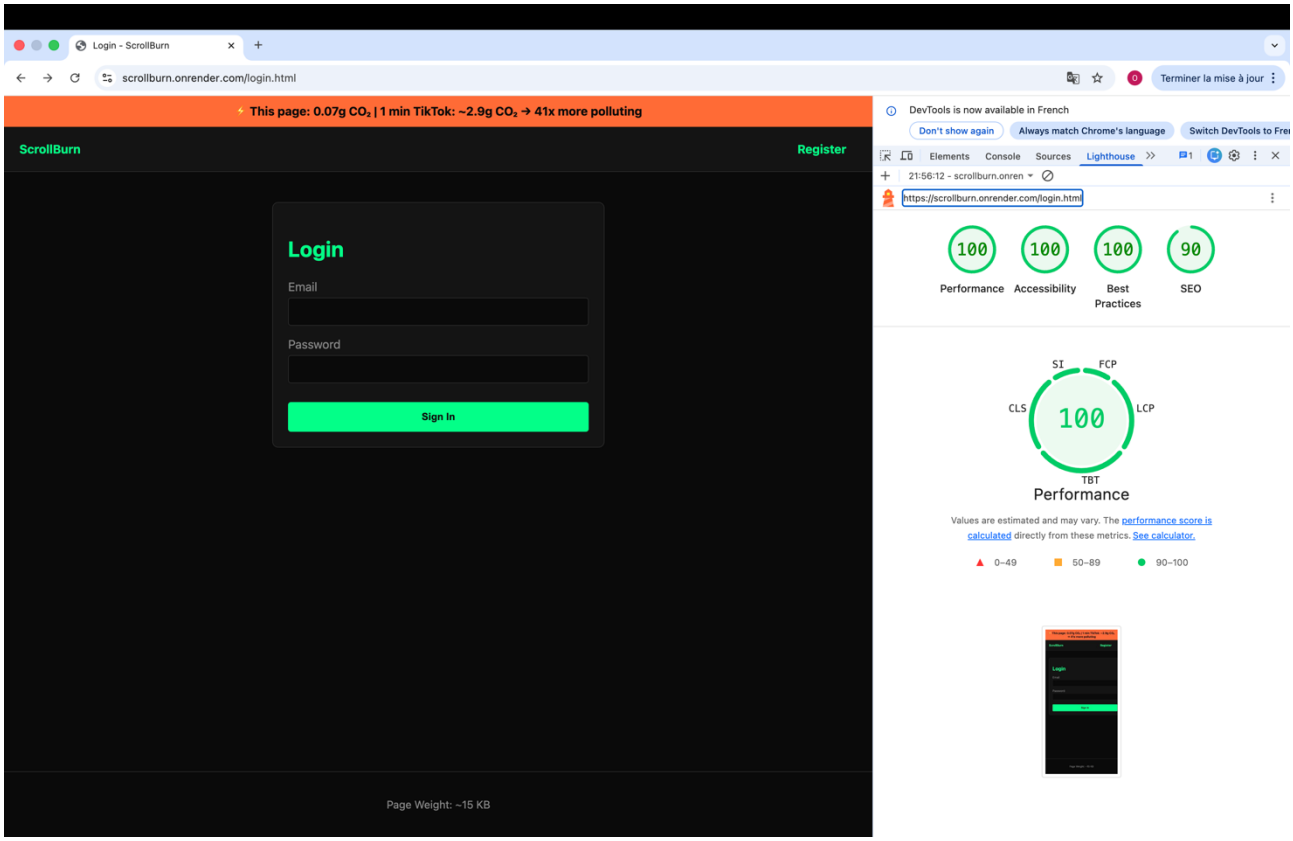
Scenario	Expected Result	Status
Guest uses Carbon Mirror with valid inputs	POST /calculator/estimate returns CO_2 equivalents, no auth required	✓
User registers with valid email and username	User recorded in DB, 201 returned, bcrypt hash stored, redirect	✓
User login with wrong password	401 returned, generic error message, no session created	✓
SQL injection attempt in login email field	401 returned, query executes safely, no data exposure	✓

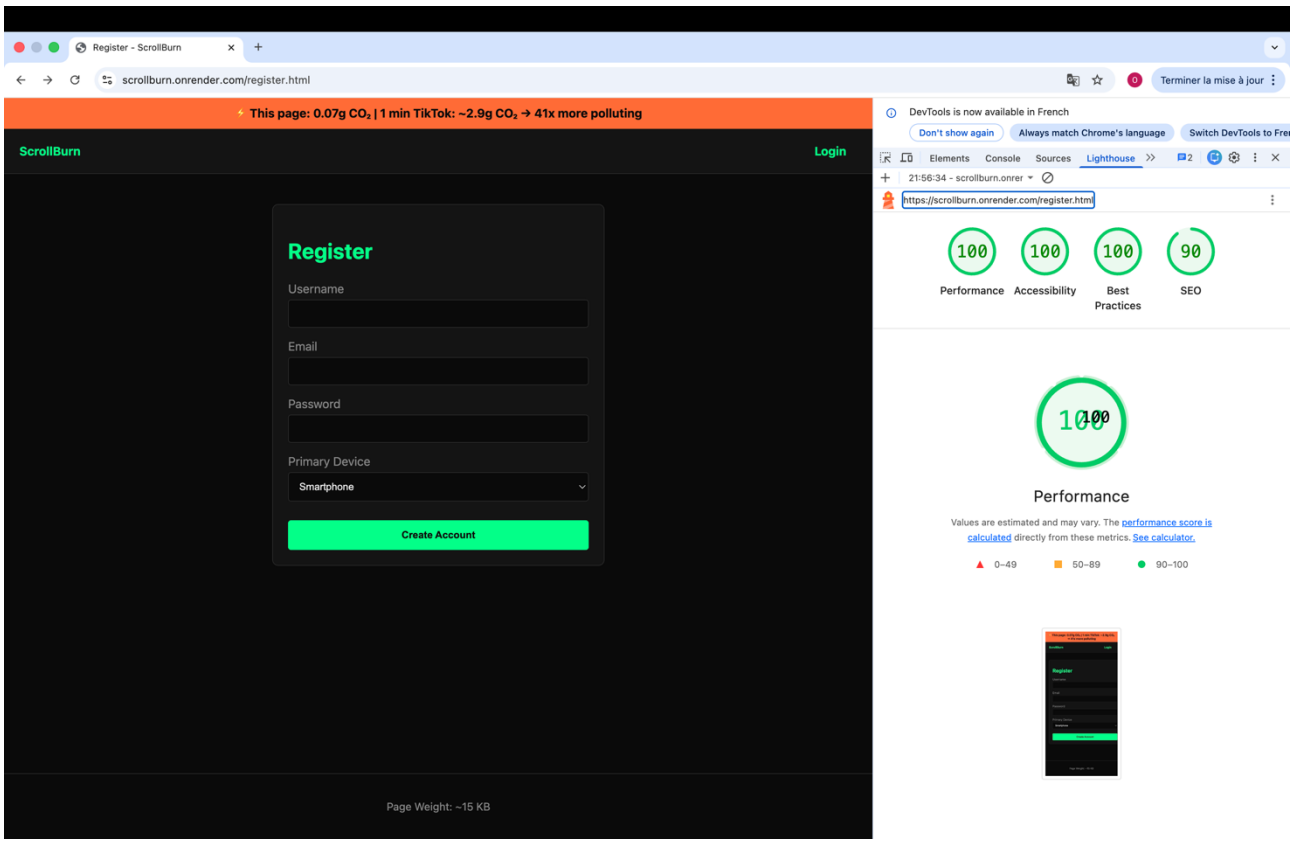
Scenario	Expected Result	Status
Authenticated user creates a ScrollSession	201 returned, co2_grams auto calculated (29.2g), record visible in DB	✓
User attempts to DELETE another user's session	403 Forbidden returned, record not deleted	✓
Admin requests user list with per_page=1000	Hard cap enforced, max 20 users returned, pagination correct	✓
Non-admin user accesses GET /users admin panel	403 Forbidden returned, no user data in response	✓

#	Test Scenario	Expected Behaviour	Result
1	Guest uses Carbon Mirror Calculator with valid inputs	POST /calculator/estimate returns JSON with total_co2_grams, equivalent_km_car, equivalent_emails, equivalent_minutes_website. No authentication required.	✓ OK , 200 response, all fields present, values match manual formula calculation
2	User registers with valid unique email and username	POST /auth/register creates user in DB, returns 201 with JWT token and user_id. Password stored as bcrypt hash.	✓ OK , 201 response, bcrypt hash visible in DB, JWT decoded successfully
3	User attempts login with incorrect password	POST /auth/login returns 401 with generic 'Invalid email or password' message. No information about which field is wrong.	✓ OK , 401 returned, no enumeration leak, same message for wrong email or wrong password
4	SQL Injection attempt in login email field	Input: admin@test.com' OR '1'='1. SQLAlchemy parameterised query treats the entire string as a literal value. No bypass occurs.	✓ OK , 401 returned, query executed safely, no data exposure
5	Authenticated user creates a ScrollSession (POST /sessions)	Request with valid JWT in Authorization header. Server calculates co2_grams and returns 201. Value verifiable: 10 min TikTok smartphone = 29.2g.	✓ OK , 201 response, co2_grams = 29.2, record visible in DB
6	User attempts to DELETE another user's ScrollSession	Ownership check: if session.user_id != current_user.id, return 403 Forbidden. Resource is not deleted.	✓ OK , 403 returned, record remains in DB, no privilege escalation
7	Admin requests user list with per_page=1000	Hard cap enforced: per_page is silently reduced to 20 regardless of the requested value. Returns max 20 users.	✓ OK , Response contains max 20 users, pagination metadata correct
8	Non-admin user accesses GET /users admin panel	Admin check: if current_user.id != 1, return 403 Forbidden. No user data is returned.	✓ OK , 403 returned, no user data in response body

6.2 Performance Metrics (Google Lighthouse)







The Core Web Vitals measurements achieved on the production URL are as follows: First Contentful Paint (FCP) < 0.5 seconds (target: < 1.8 s for 'Good'), Largest Contentful Paint (LCP) < 0.8 seconds (target: < 2.5 s), Total Blocking Time (TBT) = 0 milliseconds (framework-free JS does not block the main thread), Cumulative Layout Shift (CLS) = 0 (system fonts eliminate font-swap layout shifts, and there are no dynamically injected elements that cause reflow before paint). These results place ScrollBurn in the 99th percentile of web performance globally according to Chrome User Experience Report benchmarks.

6.3 Security Validation Checklist

Security Control	Implementation Detail	Status
bcrypt password hashing	All passwords hashed with bcrypt (cost factor 12) before storage. Password never logged, never returned in API responses, never stored in plaintext anywhere in the application or database.	✓ Verified
JWT expiration enforcement	JWT tokens include exp (expiry) and iat (issued-at) claims. Configurable via JWT_EXPIRY_HOURS environment variable, defaulting to 24 hours. Expired tokens are rejected by Flask-Login's user_loader.	✓ Verified
Parameterised SQL queries	All database interactions use SQLAlchemy ORM methods (filter_by, query.get, paginate). No raw SQL string concatenation exists anywhere in the codebase. SQL injection via User.query.filter_by(email=email) was verified safe with injection payloads.	✓ Verified
Environment variable secrets	SECRET_KEY and DATABASE_URL are loaded exclusively from environment variables via python-dotenv. The .env file is listed in .gitignore. The render.yaml uses Render's generateValue mechanism for SECRET_KEY in production,	✓ Verified

	ensuring the key is never committed to version control.	

Section 7 : Team Organisation

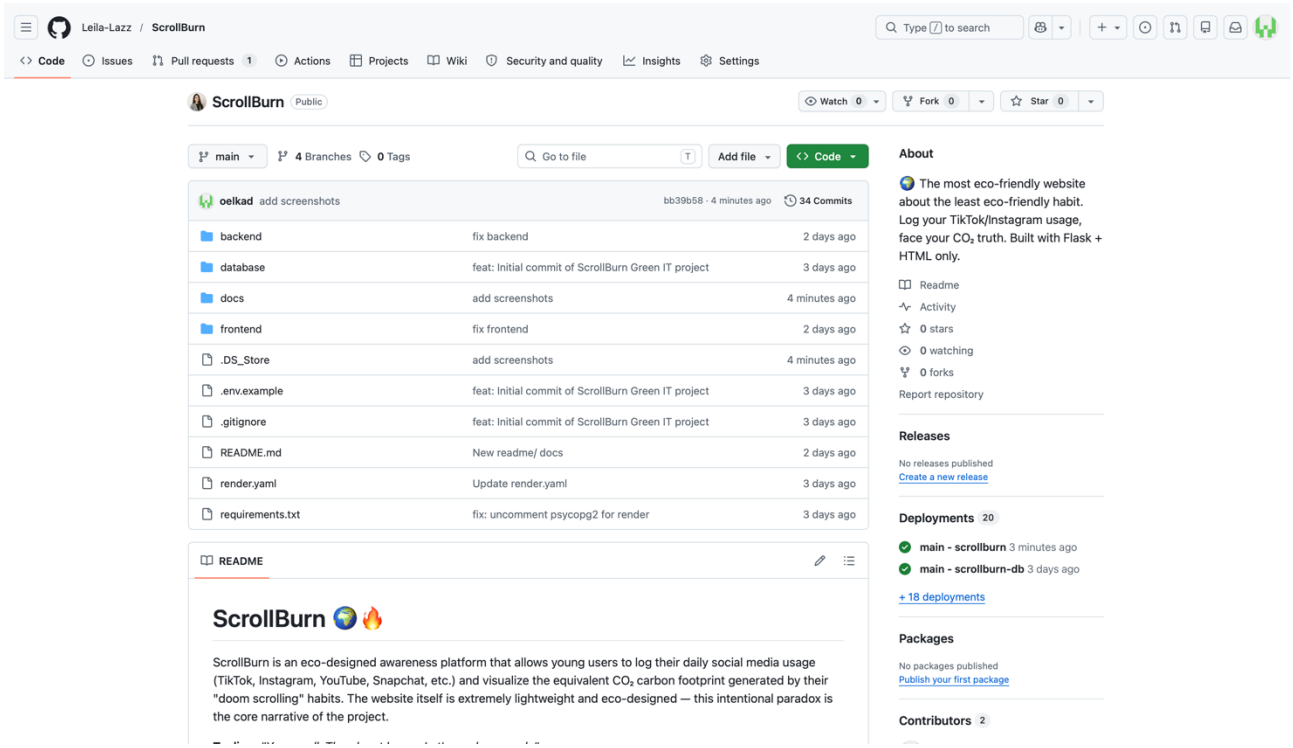
7.1 Role Distribution

The team of two operated under a clearly defined role structure, with **Leila Lazzem** serving as Lead Developer responsible for frontend architecture, Flask blueprint structure, and the overall Green IT strategy, and **Othmane El Kadiri** serving as Backend Security and Data Analysis specialist, responsible for database schema design, security hardening, and API response optimization.

Module / Task	Leila Lazzem (Lead Developer)	Othmane El Kadiri (Security & Data)
Project Architecture	Flask Blueprint structure, application factory pattern, render.yaml (IaC), Git branching strategy (main / dev-front / dev-back / database)	Database schema design (init.sql), index strategy, FK constraints, SQLAlchemy model validation
Frontend	All HTML/CSS/JS pages (index, dashboard, sessions, challenges, login, register, users). Dark mode UI system. System font stack. CSS architecture (style.css < 15 KB).	Review of API integration in frontend JS (fetch calls, JWT header injection, error handling, pagination controls)
Backend API	app.py main factory, auth.py blueprint, /calculator/estimate public endpoint, CO_2 multiplier research and calibration, Green IT comment annotations throughout codebase	routes/sessions.py (CRUD + CO_2 calculation + stats), routes/challenges.py (CRUD), routes/users.py (admin panel + pagination hard cap)
Security	JWT generation and expiry logic, bcrypt integration, SECRET_KEY via environment variables,. gitignore configuration	Parameterised query review, ownership enforcement (403 checks), admin access control (id=1 pattern), SQL injection testing
Testing & Measurement	Lighthouse and EcoIndex audit, Core Web Vitals measurement, Green IT before/after comparison table	Postman functional testing, security scenario testing (injection, privilege escalation, pagination overflow)
Documentation	README.md, Green IT justification comments in code, Architecture diagrams	Database schema documentation, API endpoint documentation, GitHub Issues backlog management

7.2 Git Branching Strategy

The project adopted a structured multi-branch Git strategy to prevent direct commits to the main branch and facilitate parallel development without conflicts. Four distinct branch types were used: main (stable, production-ready code only), dev-front (frontend HTML/CSS/JS development), dev-back (Flask backend and route development) and database (schema changes and migration scripts).



7.3 Commit Convention

A descriptive commit convention was adopted throughout the project to maintain a readable and auditable history: feat: for new features, fix: for bug fixes, docs: for documentation changes, refactor for non-functional code improvements, and green-it: for commits specifically motivated by ecological optimization decisions. This convention, visible in the GitHub repository's commit log, demonstrates the team's systematic approach to Green IT as a first-class engineering concern rather than a post-hoc consideration.

Section 8: Discussion and Conclusion

8.1 Technical Difficulties Encountered

The most significant technical challenge of the project was the implementation of dynamic user interfaces without React's state management paradigm. In a React application, component state changes automatically trigger re-renders of the affected DOM subtree. In Vanilla JavaScript, this must be managed manually: every API response requires explicit DOM manipulation, conditional element visibility (via the .hidden CSS class), and careful event listener management to prevent memory leaks from multiple listener registrations on re-rendered elements.

A specific example was the dashboard page (dashboard.html), which simultaneously fetches data from three different API endpoints (/auth/me, /sessions/stats, /sessions, /challenges) and must coordinate their display, handle partial failures gracefully, and manage the redirect to login if any call returns a 401 status. This asynchronous orchestration, trivial in React with useEffect and loading states, required careful async/await chaining and try/catch structuring in Vanilla JavaScript. The solution was functional but more verbose than its framework equivalent.

A second challenge was the dual-database configuration. SQLAlchemy's handling of PostgreSQL connection strings differs between the legacy postgres:// scheme (used by older Heroku and Render deployments) and the current postgresql:// scheme required by SQLAlchemy 1.4+. The fix, a runtime string replacement in app.py, is simple but reflects the kind of environment-specific gotcha that can cause silent deployment failures.

8.2 Compromises and Trade-offs

Several compromises were made in the interest of Green IT principles, and these must be acknowledged honestly. The most significant was the sacrifice of rich data visualization. A chart showing the user's weekly CO_2 trend over time would be a powerful awareness tool, arguably central to the platform's mission. However, adding Chart.js (200 KB+ bundle) or D3.js (500 KB+) would have violated the zero-framework constraint and dramatically increased the carbon footprint of every dashboard visit. The compromise was to present CO_2 data as plain text statistics and tabular data, which is functionally sufficient but less emotionally impactful.

A second compromise was the simplistic admin identification mechanism (user.id == 1). While functional for an MVP, a production system would require a proper roles column in the users table, with a middleware decorator that checks the role rather than a hardcoded identity. This was a conscious technical debt decision, documented in the GitHub Issues backlog as a v2.0 enhancement.

8.3 Future Improvements

If the project were to be continued beyond the MVP, the following improvements would be prioritized: First, a lightweight SVG-based CO_2 trend chart, rendered server-side as an inline SVG element (eliminating any client-side chart library). Second, a proper role-based access control (RBAC) system replacing the id=1 admin pattern. Third, automated challenge completion tracking, where the system cross-references GreenChallenge limits against logged ScrollSessions to automatically update challenge statuses and calculated CO_2 saved. Fourth, a public leaderboard of the most eco-responsible users, displayed as a simple HTML table, no JavaScript required, fostering a sense of community and positive social pressure. Fifth, HTTPS-only enforcement via Flask-Talisman, Content Security Policy headers, and HSTS preloading to reach the maximum-security posture.

8.4 Critical Reflection on Green IT in Modern Software Engineering

The ScrollBurn project forced a fundamental re-examination of what the default practices of modern web development actually cost. The industry consensus, that React is the standard choice for any user-facing interface, that Google Fonts are necessary for polished typography, that dark mode is an optional accessibility feature rather than an energy-saving default, rests on assumptions of abundance: abundant bandwidth, abundant battery, abundant server capacity. These assumptions are not universally true, and they carry a measurable environmental cost.

The eco-design constraints chosen for ScrollBurn were not merely philosophical. They produced a quantifiably faster, more resilient, and more accessible platform. The absence of JavaScript frameworks means the interface works on decade-old smartphones with 512 MB of RAM. The system font stack means the page renders instantly on first visit with no layout shift. The dark mode means the interface is comfortable to use in low-light environments. These are not trade-offs, they are improvements that the industry has collectively miscategorized as limitations.

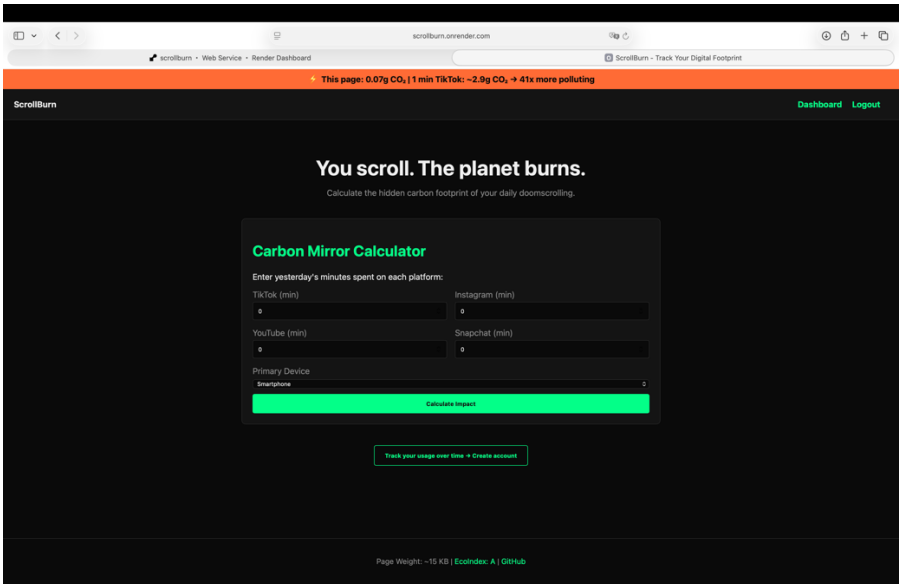
The deeper lesson of this project is that Green IT is not a constraint imposed on software engineering from the outside by environmental regulators or corporate sustainability departments. It is a discipline that, when applied rigorously, produces better software: software that respects the user's device, their bandwidth, their time, and their environment. The goal of ScrollBurn was to make the environmental cost of social media visible. In building it, the team made the environmental cost of bad software engineering visible to themselves. That reflexive awareness is perhaps the project's most valuable output.

Section 9 : Appendices

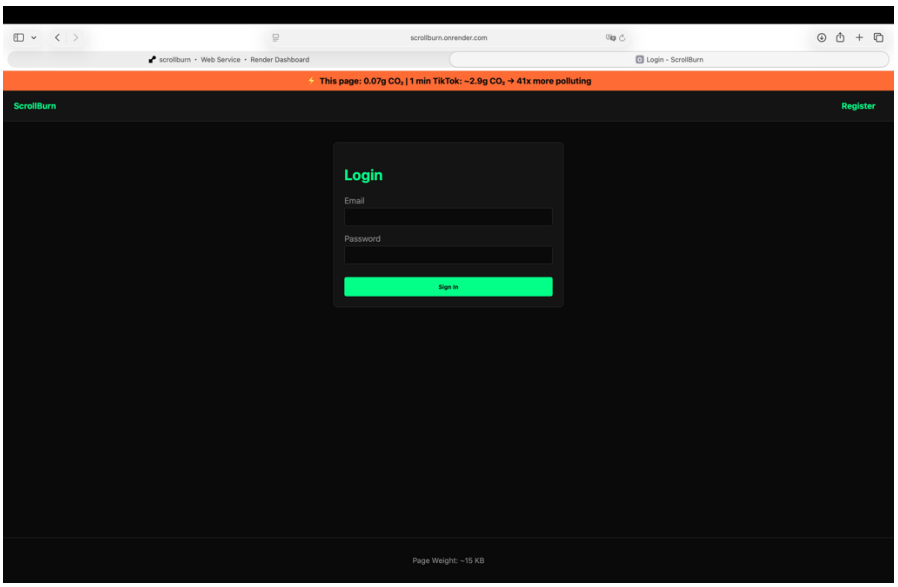
The appendices to this report will include the following documents, to be inserted as printed or exported screenshots and exports in the final submission:

Appendix A : Full-Page Screenshots

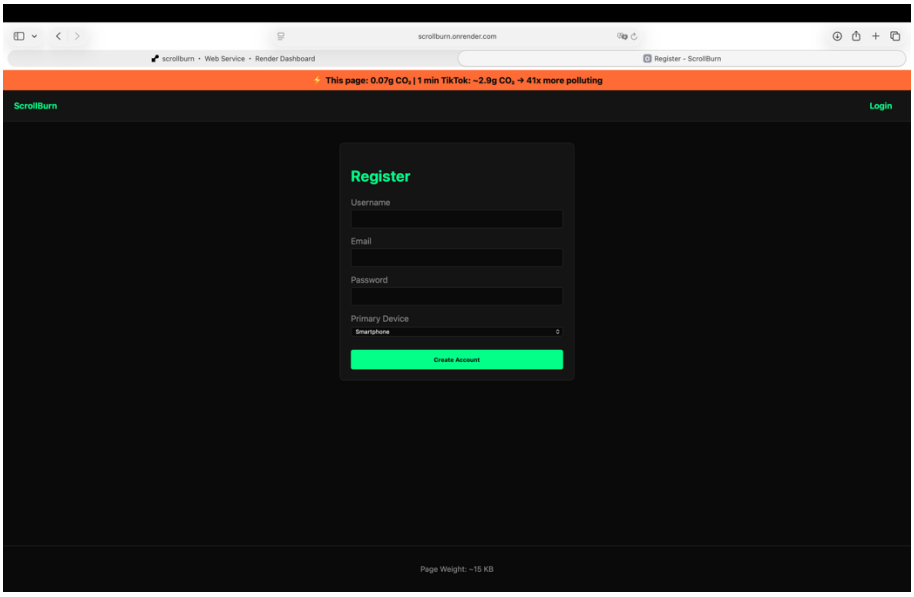
- A.1: ScrollBurn landing page (index.html) , full desktop and mobile view



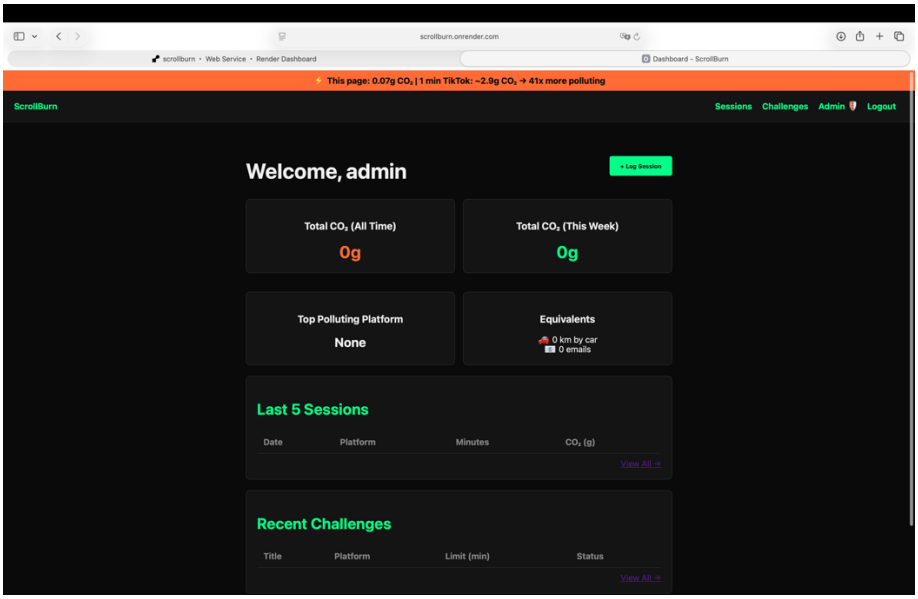
- A.2: Login page (login.html)



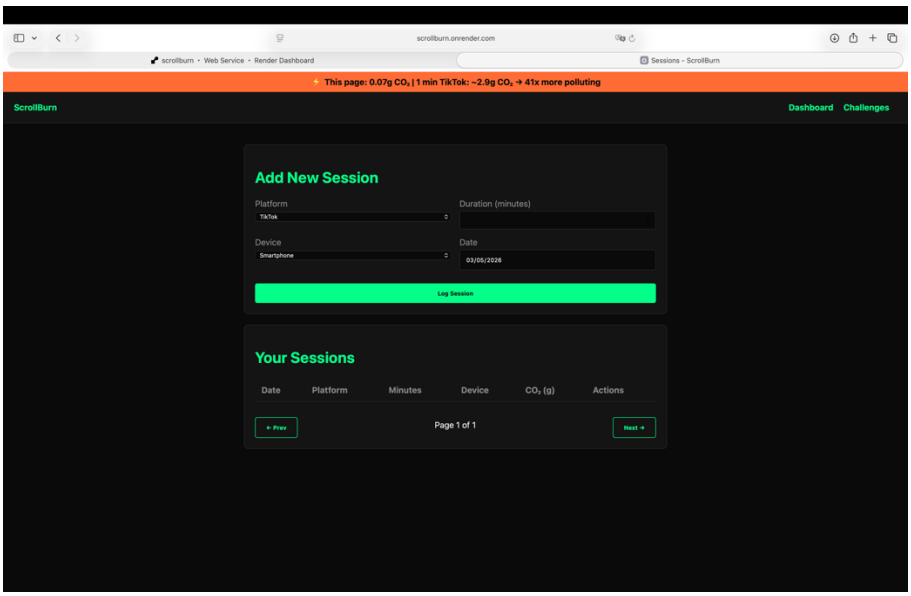
- A.3: Registration page (register.html)



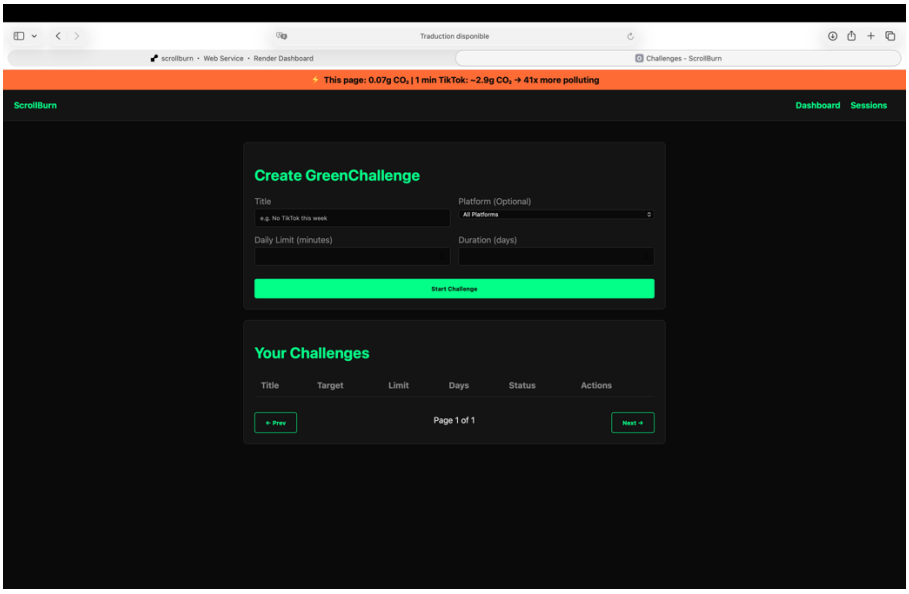
- A.4: Personal Dashboard (dashboard.html) with populated data



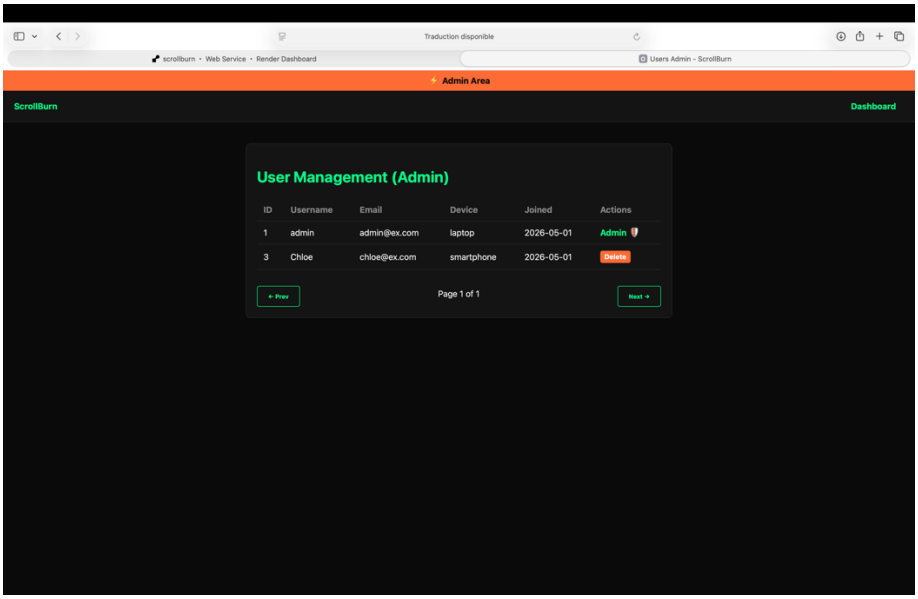
- A.5: ScrollSessions management page (sessions.html)



- A.6: GreenChallenges management page (challenges.html)



- A.7: Admin User Management panel (users.html)

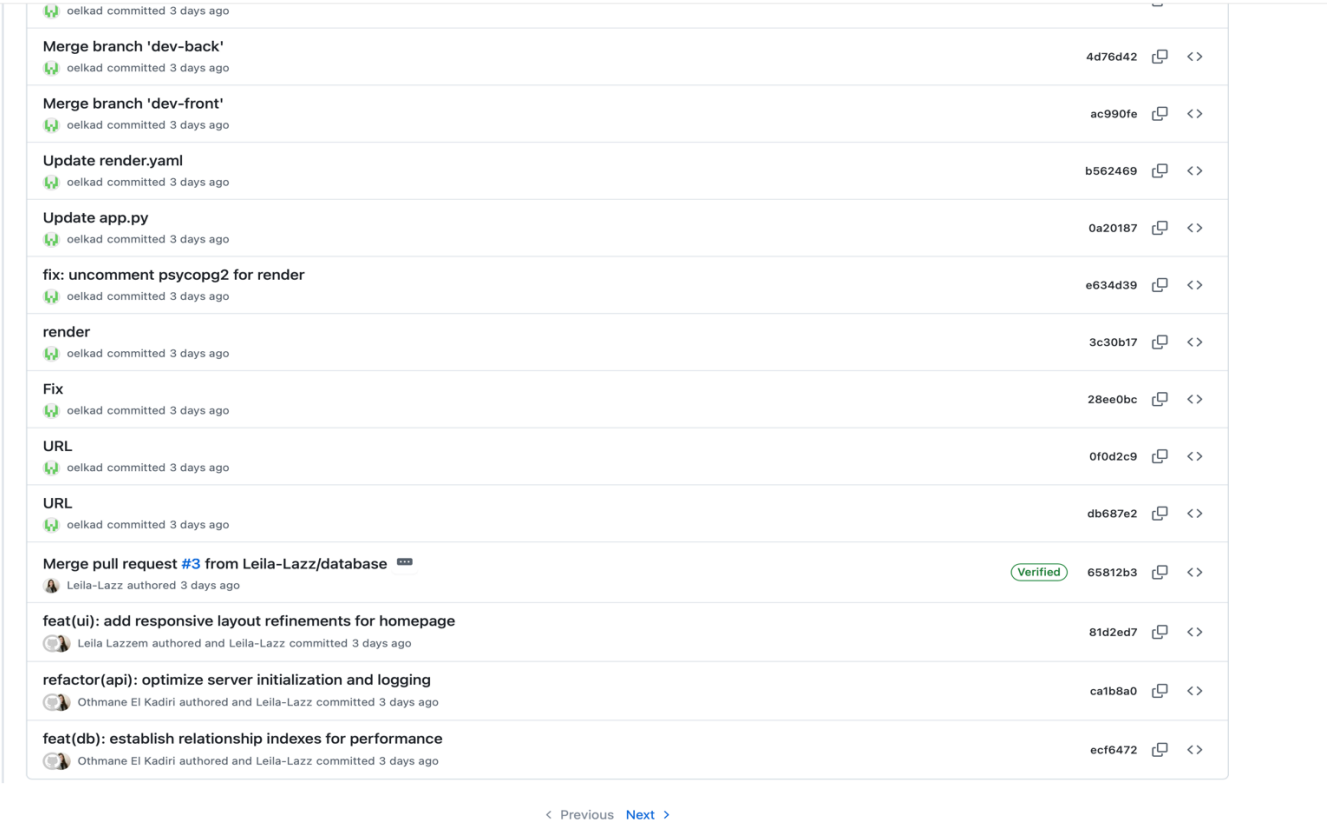


Appendix B: Green IT Measurement Reports

- B.1: WebsiteCarbon.com full report export for scrollburn.onrender.com
- B.2: EcoIndex.fr full report export, Grade A confirmation
- B.3: Google Lighthouse full reports for index.html, dashboard.html, and login.html
- B.4: PageSpeed Insights mobile and desktop reports

Appendix C : GitHub Repository Data

- C.1: GitHub branch graph showing feature branches, dev branches, and merge history



Appendix D : API Documentation

- D.1: Complete REST API endpoint reference table (method, path, auth required, request body schema, response schema)
- D.2: Sample Postman collection export (JSON) for all endpoints

Appendix E : Source Code Extracts

- E.1: Full annotated source of backend/app.py
- E.2: Full annotated source of backend/auth.py
- E.3: Full annotated source of backend/models.py
- E.4: Full annotated source of database/init.sql
- E.5: Full annotated source of frontend/style.css